



ELTE EÖTVÖS LORÁND
UNIVERSITY

Algorithms and Data Structures I.

Gábor Pusztai

Department of Algorithms And Their Applications
Faculty of Informatics
ELTE - Eötvös Loránd University, Budapest

March 1, 2026

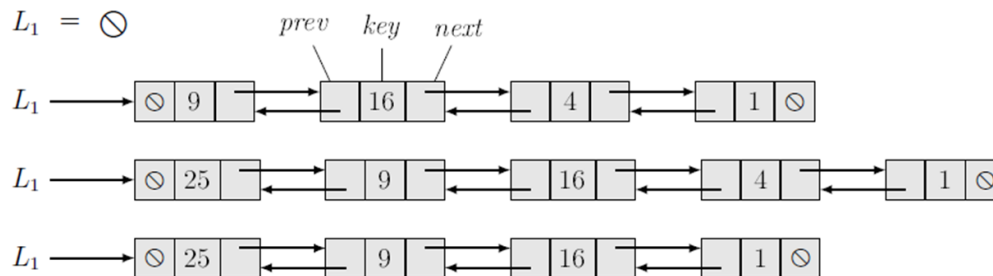
Content in this Practice I

- 1 Linked lists - Two-way (doubly)
 - Simple two-way (doubly) linked list (S2L)
 - Cyclic two-way list (C2L) with header

- 2 C2L Tasks
 - Sorted list filtered to even and odd sorted
 - Union of two C2L
 - Union and Intersection of Two C2L

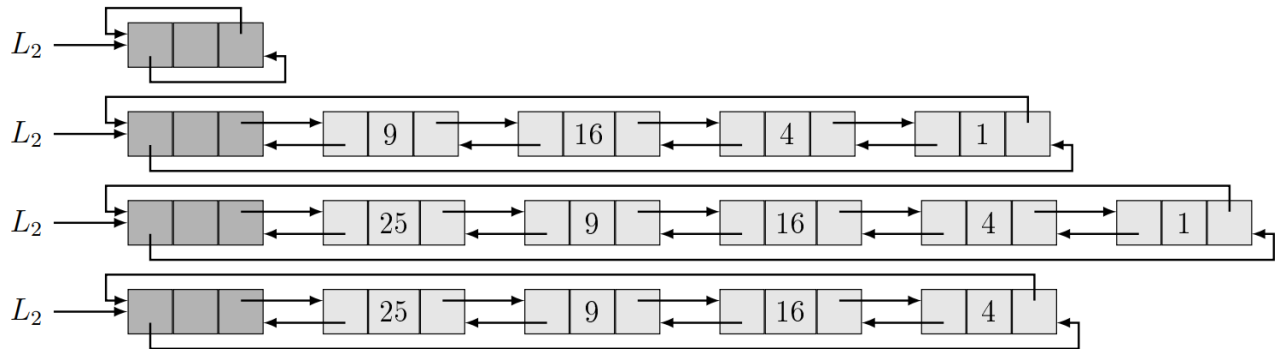
Linked lists - Two-way (doubly)

Simple two-way (doubly) linked list (S2L)



- Advantage: Easy to "maneuver" and "manipulate".
- Similar to S1L (Singly Linked List), handling the first, middle, and last elements still requires different approaches:
 - An element in the middle of a list has both a previous and a next neighbor, which means we must manage 4 pointers.
 - The first element has no previous neighbor, while the last element has no next neighbor.
 - When inserting into an empty list, the element to be inserted will have no neighbors.

Cyclic two-way list (C2L) with header



- Each element will contain the prev and the next pointer.
- The 'cyclic' aspect refers to the following:
 - The prev value of the head element always points to the last element of the list
 - The next pointer of the last element in the list, therefore points to the first element of the list

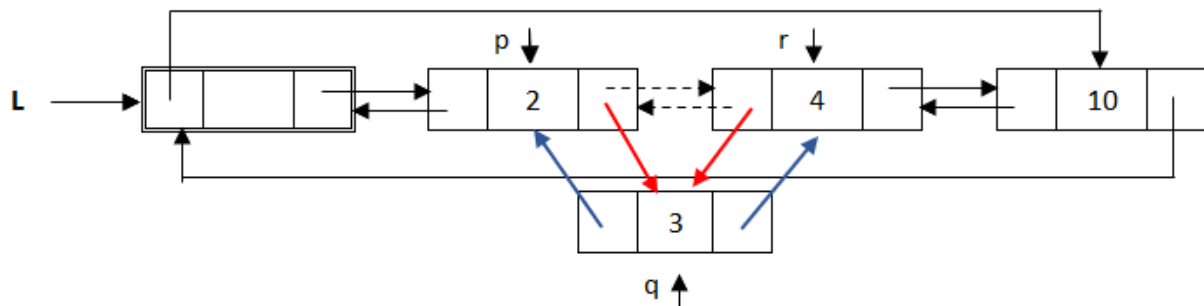
Cyclic two-way list (C2L) functions, E2 Node type

E2
+ <i>prev, next</i> : E2* // refer to the previous and next neighbour or be this
+ <i>key</i> : $\mathcal{T}$
+ E2() { <i>prev</i> := <i>next</i> := this }

$\text{precede}(q, r : \text{E2}^*)$	$\text{follow}(p, q : \text{E2}^*)$
// (*q) will precede (*r)	// (*q) will follow (*p)
$p := r \rightarrow \text{prev}$	$r := p \rightarrow \text{next}$
$q \rightarrow \text{prev} := p ; q \rightarrow \text{next} := r$	$q \rightarrow \text{prev} := p ; q \rightarrow \text{next} := r$
$p \rightarrow \text{next} := r \rightarrow \text{prev} := q$	$p \rightarrow \text{next} := r \rightarrow \text{prev} := q$

$\text{unlink}(q : \text{E2}^*)$
// remove (*q)
$p := q \rightarrow \text{prev} ; r := q \rightarrow \text{next}$
$p \rightarrow \text{next} := r ; r \rightarrow \text{prev} := p$
$q \rightarrow \text{prev} := q \rightarrow \text{next} := q$

Cyclic two-way list (C2L) functions - precedence



$\text{precede}(q, r : E2^*)$

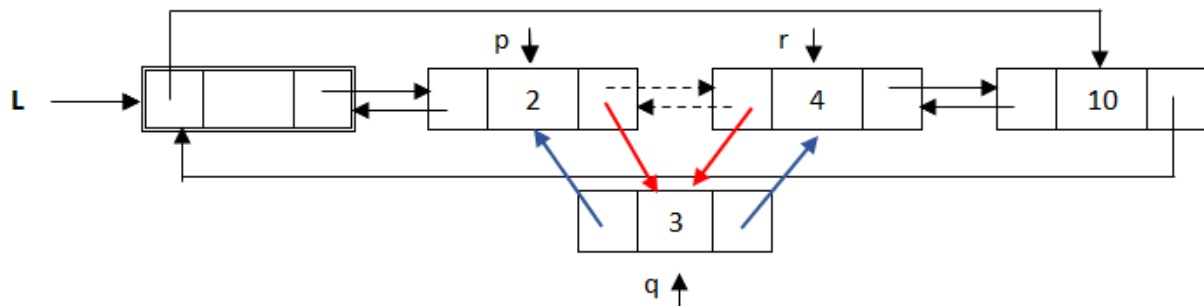
// (*q) will precede (*r)

$p := r \rightarrow \text{prev}$

$q \rightarrow \text{prev} := p ; q \rightarrow \text{next} := r$

$p \rightarrow \text{next} := r \rightarrow \text{prev} := q$

Cyclic two-way list (C2L) functions - follow



$\text{follow}(p, q : E2^*)$

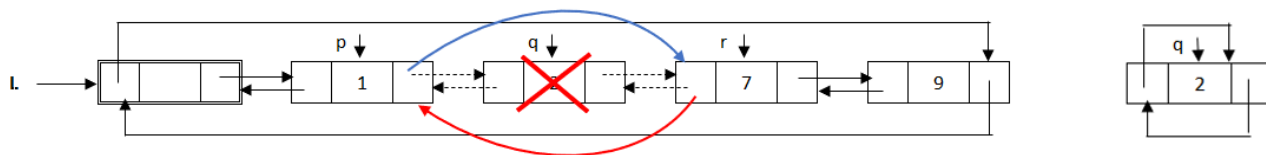
// $(*q)$ will follow $(*p)$

$r := p \rightarrow \text{next}$

$q \rightarrow \text{prev} := p ; q \rightarrow \text{next} := r$

$p \rightarrow \text{next} := r \rightarrow \text{prev} := q$

Cyclic two-way list (C2L) functions - unlink



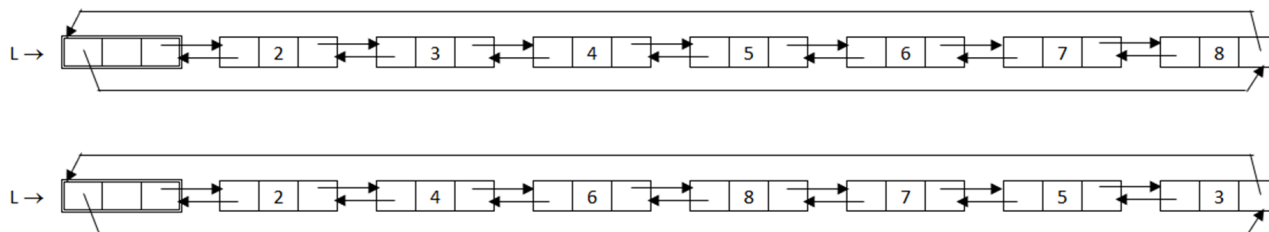
$\text{unlink}(q : E2^*)$

$// \text{ remove } (*q)$
$p := q \rightarrow \text{prev} ; r := q \rightarrow \text{next}$
$p \rightarrow \text{next} := r ; r \rightarrow \text{prev} := p$
$q \rightarrow \text{prev} := q \rightarrow \text{next} := q$

C2L Tasks

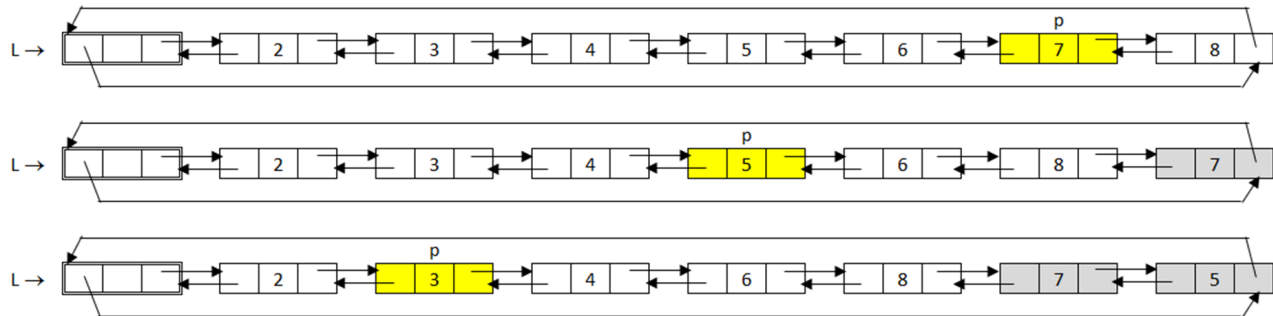
Sorted list filtered to even and odd sorted

Sorted list filtered to even and odd sorted



- Let L point to an ordered C2L list containing natural numbers.
- Let's reorder the list so that at the 'front' of the list there are only even numbers, and at the 'end' there are only odd numbers.
- Note: Both 'parts' of the list should remain ordered.

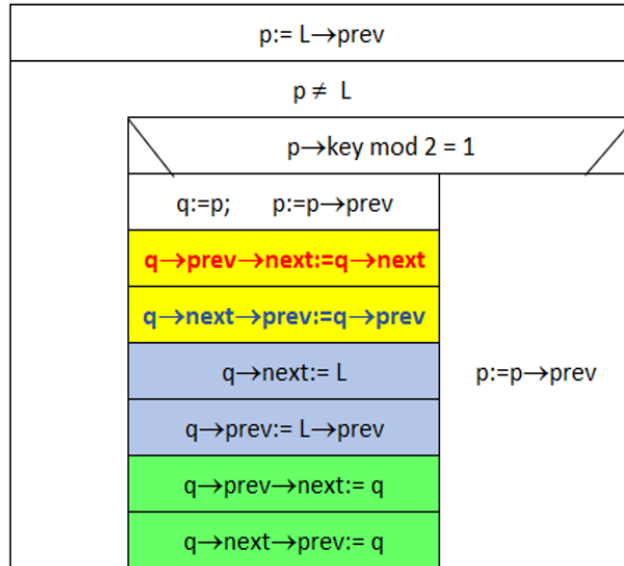
Sorted list filtered to even and odd sorted



- Iterate backwards through the list using p .
- If the key of element p is odd, unlink the element and re-link it to the end of the list (before the head element).
- We will use an auxiliary pointer q for the unlinking; we move p to the next position (in the backward direction) before performing the unlinking.

Sorted list filtered to even and odd sorted

ReorderC2L(L:E2*)



We only have to do anything if p is odd

Yellow: Linking out the odd element

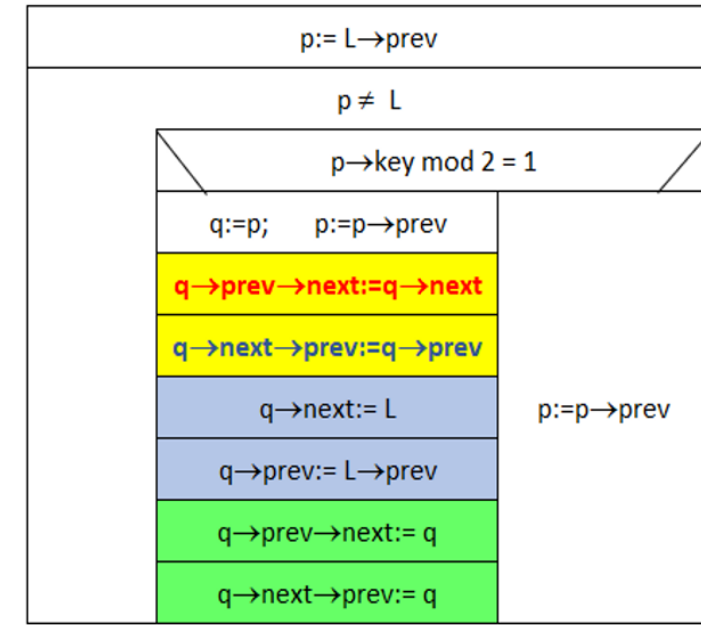
Blue: Inserting q before the header.

Green: Setting the neighbours' pointers

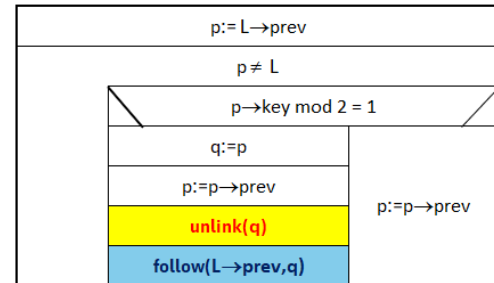
33

Sorted list filtered to even and odd sorted - simplified

ReorderC2L(L:E2*)



Simplified ReorderC2L(L:E2*)

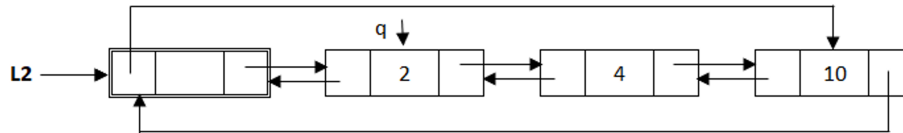
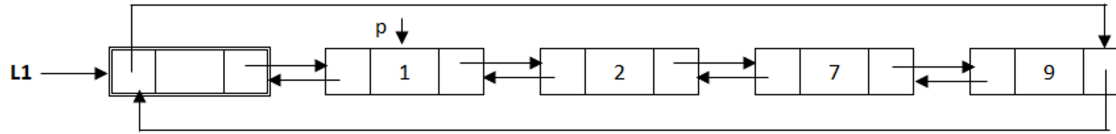


Union of two C2L

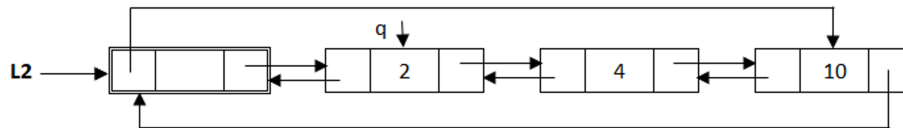
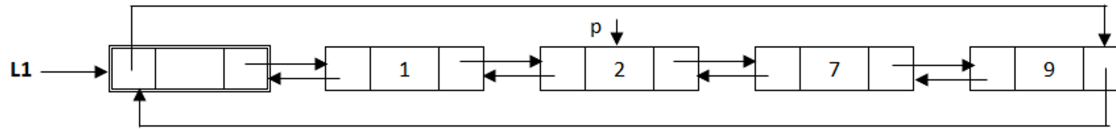
Union of two C2L

- Combining two lists is a frequent task to solve, and not just for job interviews. The sorted nature of these lists plays an essential role in the efficiency of these algorithms. Since we are no longer working with arrays, inserting an element does not require shifting others.
- Usually we work with lists containing a header (H1L, C2L).
- An example: Let L1, and L2 each point to different sorted C2L. Let the keys also be unique (the list therefore can be considered as a set). Let the output be L1 and the list it corresponds to be the union of the two sets.
- Idea: Each list has a pointer iterating through its elements. Then, we have 3 cases

Union of two C2L - case 1

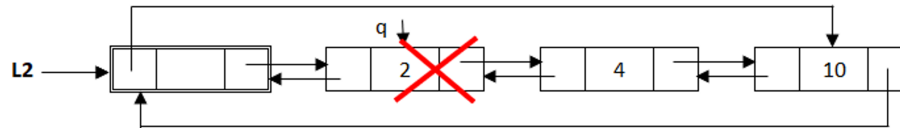
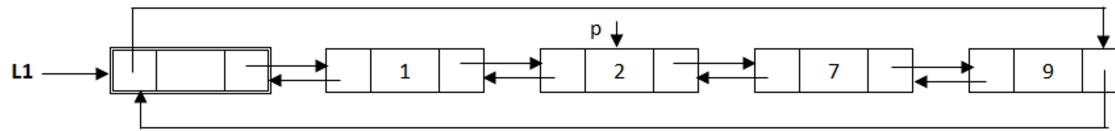


P keeps going, but q remains in place

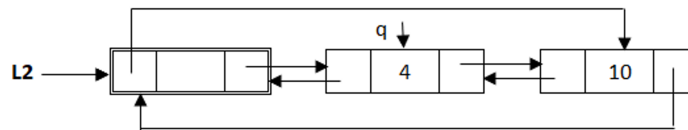
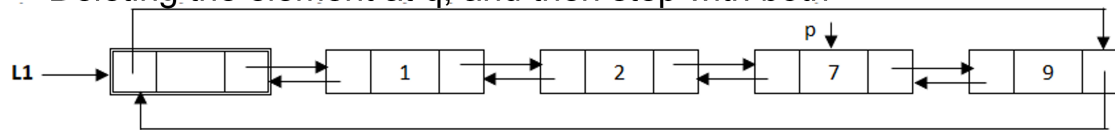


- (1) The current element in L1 is smaller than the element in L2, we step with the pointer in L1, but remain stationary in L2.

Union of two C2L - case 2

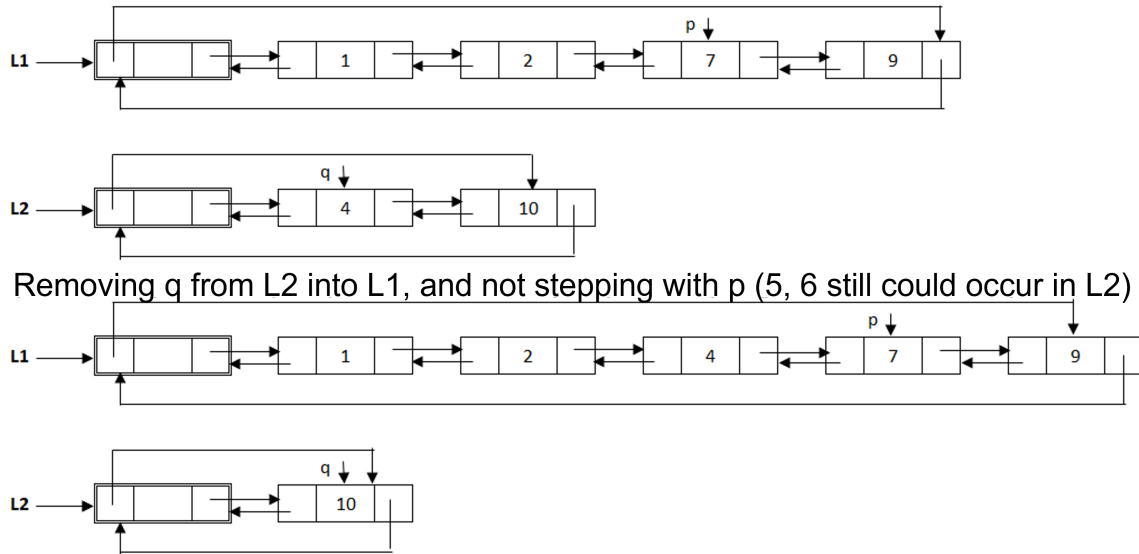


Deleting the element at q , and then step with both



- (2) The value of the current element in L1 equals to the current element in L2. We remove the element from L2 and move with both pointers.

Union of two C2L - case 3



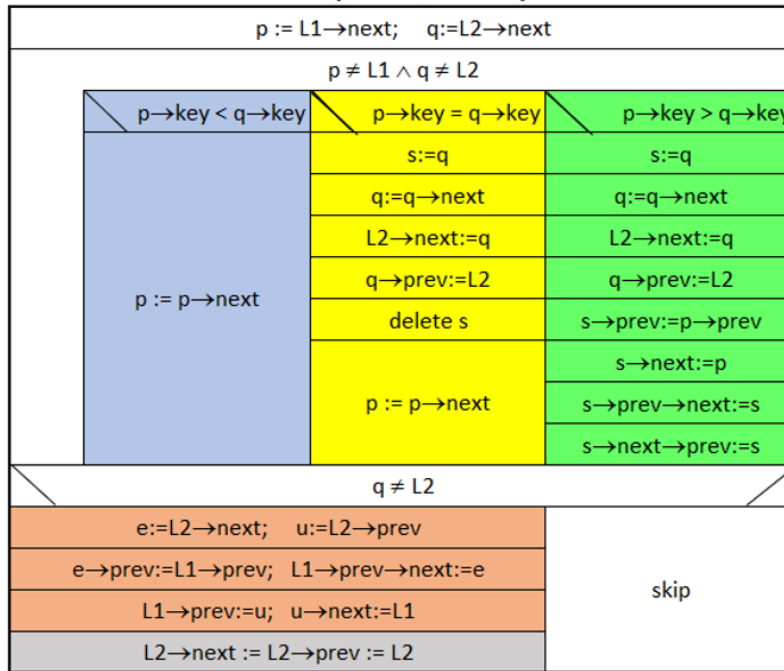
- (3) L1 has a larger element at p , than the element at q . Then we insert the element at q into L1 by removing it from L2, and step with q , while not moving with p .

Union of two C2L - Reaching the end of a list

- If $p=L1$ then we reached end of L1, otherwise if $q=L2$, then the end of L2. (It could occur at the same time in cases where we step with both.)
- Reaching the end of L2 means we are done, there are no more elements to potentially insert into L1. Otherwise, if we reach the end of L1, that is equivalent to saying that there are still elements in L2 that are not present in L1. Meaning we need to copy the remaining L2 into L1. This can be done one by one, but in constant time too!
- Note: At any point in the algorithm the following is true: $q=L2 \rightarrow next$. This means that we don't need the q variable when we implement it, but for readability, we will still use it for now.

Union of two C2L

Unio(L1: E2*, L2: E2*)



- Blue:
 - Current element in L1 is smaller than the current element in L2: Just step with p.
- Yellow:
 - Elements are equal. This means that L1 already contains that element, so we just have to remove the element at q from L2, and set it to the next element.
- Green:
 - Found an element which is not present in L1, meaning we have to insert it from L2 to L1. This is the same as the previous except this time we not only remove q from L2, but also insert it into L1, and not step with p.
- Brown:
 - Append the remaining elements from L2 to L1.
- Note:
 - The order of some instructions is essential in order to work properly, but not all. Setting the prev before the next doesn't make a difference, but we cannot set the values of the neighbouring elements before we set our owns.

Union Two C2L - Simplified Version

Unio(L1: E2*, L2: E2*)

$p := L1 \rightarrow next; \quad q := L2 \rightarrow next$

$p \neq L1 \wedge q \neq L2$

$p \rightarrow key < q \rightarrow key$	$p \rightarrow key = q \rightarrow key$	$p \rightarrow key > q \rightarrow key$
$p := p \rightarrow next$	$s := q$	$s := q$
	$q := q \rightarrow next$	$q := q \rightarrow next$
	$L2 \rightarrow next := q$	$L2 \rightarrow next := q$
	$q \rightarrow prev := L2$	$q \rightarrow prev := L2$
	delete s	$s \rightarrow prev := p \rightarrow prev$
	$p := p \rightarrow next$	$s \rightarrow next := p$
		$s \rightarrow prev \rightarrow next := s$
		$s \rightarrow next \rightarrow prev := s$

$q \neq L2$

$e := L2 \rightarrow next; \quad u := L2 \rightarrow prev$

$e \rightarrow prev := L1 \rightarrow prev; \quad L1 \rightarrow prev \rightarrow next := e$

$L1 \rightarrow prev := u; \quad u \rightarrow next := L1$

$L2 \rightarrow next := L2 \rightarrow prev := L2$

skip

Unio(L1: E2*, L2: E2*)

$p := L1 \rightarrow next; \quad q := L2 \rightarrow next$

$p \neq L1 \wedge q \neq L2$

$p \rightarrow key < q \rightarrow key$	$p \rightarrow key = q \rightarrow key$	$p \rightarrow key > q \rightarrow key$
$p := p \rightarrow next$	$s := q$	$s := q$
	$q := q \rightarrow next$	$q := q \rightarrow next$
	unlink(s)	unlink(s)
	delete s	precede(s,p)
	$p := p \rightarrow next$	

$L2 \rightarrow next \neq L2$

$s := L2 \rightarrow next$

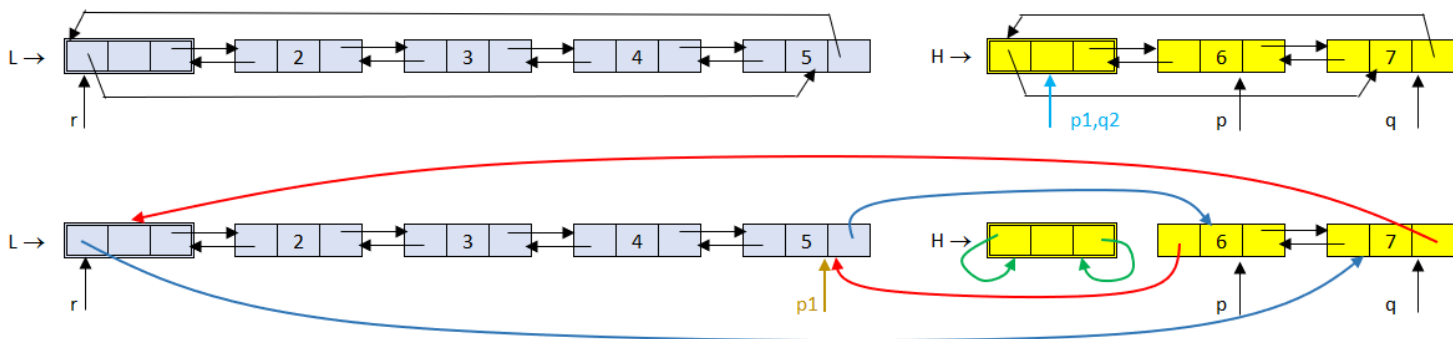
unlink(s)

follow(L1 $\rightarrow$ prev, s)

Union Two C2L - More Efficiently

- We can make our solution more efficient: when exiting the main loop, if there are still elements remaining in the L2 list, we do not move them into L1 using a loop. Instead, we link the entire L2 segment to the end of L1 in constant time by its endpoints.
- The splice($p, q, r : E2^*$) is an elementary list operation that removes a specific $[p, \dots, q]$ segment of a C2L and inserts the removed segment before a given element $*r$ of a C2L. (Precondition: the $p..q$ segment must not contain the head element, p must precede q ($p=q$ is possible), and $*r$ must not be an element of the $p..q$ segment, although r can be an element of a different C2L list.)

Union Two C2L - More Efficiently



$\text{append}(L, H : E2^*)$

$H \rightarrow \text{next} \neq H$	
$\text{splice}(H \rightarrow \text{next}, H \rightarrow \text{prev}, L)$	SKIP

$\text{splice}(p, q, r : E2^*)$

$p1 := p \rightarrow \text{prev} ; q2 := q \rightarrow \text{next}$
$p1 \rightarrow \text{next} := q2 ; q2 \rightarrow \text{prev} := p1$
$p1 := r \rightarrow \text{prev}$
$p \rightarrow \text{prev} := p1 ; q \rightarrow \text{next} := r$
$p1 \rightarrow \text{next} := p ; r \rightarrow \text{prev} := q$

Union Two C2L - More Efficiently

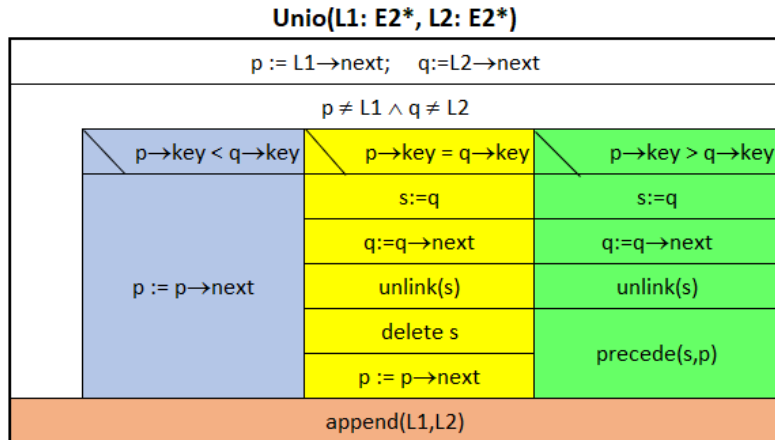
$\text{splice}(p, q, r : E2^*)$

$p1 := p \rightarrow prev ; q2 := q \rightarrow next$
$p1 \rightarrow next := q2 ; q2 \rightarrow prev := p1$
$p1 := r \rightarrow prev$
$p \rightarrow prev := p1 ; q \rightarrow next := r$
$p1 \rightarrow next := p ; r \rightarrow prev := q$

- $p1$ is the left neighbor of p , $q2$ is the right neighbor of q
- Link elements $p1$ and $q2$ together (unlinking the segment)
- $p1$ is the left neighbor of r ; we insert the $p..q$ segment between $p1$ and r
- Connecting elements p and q with elements $p1$ and r
- Connecting elements $p1$ and r with elements p and q

- With this, we can define the append operation, which appends all elements of a non-empty C2L list H to the end of a C2L list L .

Union Two C2L



- Let's examine the operational complexity of our Union algorithm:
- Let the length of list L1 be n , and the length of list L2 be m .
- The algorithm will iterate through one of the lists regardless, so:
- $mT(n, m) = \theta(n)$ in the case where every element of L1 is smaller than the first element of L2.
- $mT(n, m) = \theta(m)$ in the case where every element of L2 is smaller than the first element of L1.
- $MT(n, m) = \theta(n + m)$. For example, it performs exactly $n + m$ iterations if there are no identical elements, and the second-to-last element of L1 is smaller, while the last is larger, than the last element of L2.

Union and Intersection of Two C2L

Union and Intersection of Two C2L

- Let Hu and Hi be strictly monotonically increasing sorted C2L!
- Write the `unionIntersection( $Hu, Hi : E2^*$ )` procedure, which re-links the appropriate elements of list Hi into list Hu , such that list Hu produces the union of the original lists (as sets), while the intersection remains in list Hi !
- Do not allocate or deallocate list elements; solve the task only by re-linking the existing list elements!
 $MT(n_u, n_i) \in \theta(n_u + n_i)$, where n_u is the length of C2L Hu and n_i is the length of C2L Hi . Both lists must remain strictly monotonically increasing sorted C2L!

Union and Intersection of two C2L

$$H_u \rightarrow [] - [2] - [4] - [6] - [] \leftarrow H_u$$

$$H_i \rightarrow [] - [1] - [4] - [5] - [8] - [9] - [] \leftarrow H_i$$

$$H_u \rightarrow [] - [1] - [2] - [4] - [6] - [] \leftarrow H_u$$

$$H_i \rightarrow [] - [4] - [5] - [8] - [9] - [] \leftarrow H_i$$

$$H_u \rightarrow [] - [1] - [2] - [4] - [6] - [] \leftarrow H_u$$

$$H_i \rightarrow [] - [4] - [5] - [8] - [9] - [] \leftarrow H_i$$

$$H_u \rightarrow [] - [1] - [2] - [4] - [6] - [] \leftarrow H_u$$

$$H_i \rightarrow [] - [4] - [5] - [8] - [9] - [] \leftarrow H_i$$

$$H_u \rightarrow [] - [1] - [2] - [4] - [5] - [6] - [] \leftarrow H_u$$

$$H_i \rightarrow [] - [4] - [8] - [9] - [] \leftarrow H_i$$

$$H_u \rightarrow [] - [1] - [2] - [4] - [5] - [6] - [] \leftarrow H_u$$

$$H_i \rightarrow [] - [4] - [8] - [9] - [] \leftarrow H_i$$

$$H_u \rightarrow [] - [1] - [2] - [4] - [5] - [6] - [8] - [] \leftarrow H_u$$

$$H_i \rightarrow [] - [4] - [9] - [] \leftarrow H_i$$

$$H_u \rightarrow [] - [1] - [2] - [4] - [5] - [6] - [8] - [9] - [] \leftarrow H_u$$

$$H_i \rightarrow [] - [4] - [] \leftarrow H_i$$

unionIntersection($H_u, H_i : E2*$)

$q := H_u \rightarrow next ; r := H_i \rightarrow next$		
$q \neq H_u \wedge r \neq H_i$		
$q \rightarrow key < r \rightarrow key$	$q \rightarrow key > r \rightarrow key$	$q \rightarrow key = r \rightarrow key$
$q := q \rightarrow next$	$p := r$	$q := q \rightarrow next$
	$r := r \rightarrow next$	
	unlink(p)	$r := r \rightarrow next$
	precede(p, q)	
$r \neq H_i$		
$p := r ; r := r \rightarrow next ; unlink(p)$		
precede(p, H_u)		

Thank you for your attention!